HOTEL RECEPTIONIST SYSTEM

INTRODUCTION

Background:

Small local stay-inns/hotels/resorts usually do not have a digital solution to control room reservations, arrivals, and departures. These are usually done manually, and there is a high possibility of error that can lead to overbookings, bad customer experience, and data loss. A centralized database system can simplify front desk operations and enhance the quality of service.

Problem Statement:

Receptionists in small hotels struggle to keep track of guest check-ins, room availability, and payment by way of pen-and-paper or spreadsheets. This hinders the efficiency and slows down the recording process. A simple yet effective and efficient system is needed to handle such an important task.

Scope:

The main goal of this data management system is to handle guest registrations, keep track of the availability of rooms, assist in check-in and check-out processes, and record payment transactions. This system is meant to be used by small local hotels with fewer than 100 rooms.

- Receptionists can check in guests and assign rooms.
- Receptionists can check out guests and record payments.
- Create bookings for guests and record guests' information.
- Tracks room availability, all guests’ bookings, and stay duration.

Target Users:

Hotel Receptionists

Hotel Managers / Owner (Admin)

PROJECT OBJECTIVES

1. **Room Management System and Check-in/out System**
    a. Update room status automatically upon the guests' check-in and check-out
    b. Link the bookings to the room, hence creating a room availability tracking function.
    c. Records guests' check-ins and check-outs.
    d. Admin can add rooms and designate their price.
2. **Guest Management System and Booking System**
    a. Shows the list of all the guests and their information.
    b. Create and view guests' bookings and reservations.
3. **Staff Management System**
    a. Admin can add staff who will manage guests' check-ins and check-outs, as well as the bookings.

BUSINESS RULES

1. **Room Type and Price Definition**
    a. Constraints
        i. Price must be a positive decimal value greater than 0.
        ii. Only the administrator can add new rooms.
    b. Conditions
        i. A room cannot be created without both type and price specified.
        ii. A room cannot be deleted once occupied or has a booking history.
        iii. Price changes require an administrator.
2. **Room Status Tracking**
    a. Constraints
        i. Status must be one of: **Vacant, Reserved, Occupied**
    b. Conditions
        i. The room status will be updated once a guest books the room, checks in, or checks out.

3. **Booking Overlap Prevention**
    a. Constraints
        i. No two confirmed bookings can overlap for the same room.
        ii. The system must check availability in real-time before confirmation.
        iii. Past dates cannot be selected for new bookings.
    b. Conditions
        i. Cancelled bookings immediately free up room availability.
4. **Booking Status Management**
    a. Constraints
        i. Status must be one of: **Confirmed, Checked-In, Checked-Out, or Cancelled.**
        ii. Cancelled bookings can be deleted, but cannot be reactivated (a new booking is required)
    b. Conditions
        i. Automatic status updates at check-in/check-out date.
        ii. Status history must be maintained for audit purposes.
5. **Guest and Room Association**
    a. Constraints
        i. Every booking must link to exactly one room.
    b. Conditions
        i. Guest information must be complete before booking and check-in.
6. **User Access Management**
    a. Constraints
        i. Receptionists only have limited to booking, check-ins and out, and guest management modules.
    b. Conditions
        i. Admins have full system access, which includes all modules and functions.

DATABASE MODELS

ERD: See documentation/diagrams/erd.png

Relational Model: See documentation/diagrams/rm.png

PROJECT OVERVIEW

This project is a database application designed to support local hotel front desk operations. It helps receptionists to manage check-ins and check-outs, guests, room availability searches, and payment procedures.

SETUP INSTRUCTIONS

1. **Clone or download the repository**
2. **Install the required Python packages**

   ```
   pip install flask flask-sqlalchemy mysql-connector-python
   python-dotenv werkzeug
   ```

3. **Set up the database**
   - Start XAMPP and ensure the MySQL service is running
   - Run the database creation script:

     ```
     python create_database.py
     ```

   - Or manually import `schema.sql` through phpMyAdmin
   - (Optional) Import `initial_data.sql` through phpMyAdmin
4. **Set up environment variables**
   - Create a `.env` file in the root directory with the following content:

     ```
     # Flask configuration
     SECRET_KEY="groupthree"
     # Database configuration
     DB_USER=root
     DB_PASSWORD=""
     DB_HOST=localhost
     DB_PORT=3306
     DB_NAME=CCCS105
     ```

Replace `"groupthree"` with a secure random string

Replace `""` with your MySQL password

TEAM MEMBERS

Alba, John Raymond S. (Documentation)

David, Kenji Nathaniel R. (Documentation)

Doroin, Ivy M. (Documentation)

Orias, Mark Joseph C. (Programmer & Documentation)

DEPENDENCIES

mysql-connector-python==8.0.33

Flask==3.0.2

Flask-SQLAlchemy==3.1.1

python-dotenv==1.0.1

Werkzeug==3.0.1

RUNNING INSTRUCTIONS

Open the Project in Visual Studio Code (VS Code)

- Launch VS Code.
- Click on **File** → **Open Folder...** and select the folder where you cloned or extracted the project.
- Once the project is open, VS Code might prompt you to install the Python extension. If it does, click **Install**.

Select the Python Interpreter

- Press `Ctrl+Shift+P (or Cmd+Shift+P on Mac)` to open the Command Palette.
- Search for and select: **Python: Select Interpreter.**

- Choose the interpreter or virtual environment where you installed the required packages.

  Example: `Python 3.x.x (venv)` or the path to your global Python installation.

Run the Application

- Open the `run.py file` from the file explorer in VS Code
- Right-click inside the file and choose **Run Python File in Terminal**.

Access the application

The application will be available at `http://127.0.0.1:5000/`

Admin access:

Username: admin

Password: admin123

Receptionist access:

Username: receptionist

Password: receptionist123

DATABASE SETUP

1. **Create MySQL database:**
   - `CREATE DATABASE CCCS105;`
2. **Configure database connection:**
   - Database Name: CCCS105
   - Username: root
   - Password: (no password)
   - Host: localhost
   - Port: 3306
3. **Import schema and initial data:**
   - Open MySQL Command Line Client
   - Use the following commands:

     ```
     mysql -u root CCCS105 < database/schema.sql
     ```

     ```
     mysql -u root CCCS105 < database/initial_data.sql
     ```